```
Examples:
| message                       | workflow               |
| sydney_melbourne.txt          | domestic-direct        |
| sydney_melbourne-hobart.txt   | domestic-transfer      |
| sydney_wellington.txt         | international-direct   |
| sydney_toronto.txt            | international-transfer |
```

In both of these scenarios, there will be a web page that monitors the state of registered bags, but that's not really what's being tested here. Testing application code is faster and more robust than exercising the UI, and the bulk of the business rules can often be safely tested directly against the application code, but such testing may inspire less confidence in the results. A small number of additional end-to-end UI tests can alleviate these doubts and verify that the baggage details are correctly displayed.

These scenarios are typical of many business-focused acceptance criteria, especially scenarios involving file processing, message routing, and so forth. In general, acceptance criteria should be of the end-to-end variety. They should, at least in principle, exercise all of the components of the system and verify how they work together. This is usually a more reliable way to build confidence in the acceptance tests. But testing specific business rules can often be done more efficiently within an isolated context, especially when an acceptance criterion is focused on a specific calculation or operation, with well-defined input parameters and easily identifiable outcomes.

The Cucumber step definitions for the first scenario might look like this:

```
public class BaggageRegistrationSteps {

    String message;
    BaggageRegistration baggageRegistration;
    RegistrationService service;

    @Given("^a baggage registration message:$")
    public void a_baggage_registration_message(String messageText)
      throws Throwable {
        this.message = messageText;
    }

    @When("^the baggage registration is processed$")
    public void the_baggage_registration_is_processed() throws Throwable {
        service = new RegistrationService();
        baggageRegistration = service.registerBaggage(message);
    }

    @Then("^the registration details should be:$")
    public void registration_details_should_be(
                              List<BaggageRegistration> expectedDetails)
      throws Throwable {
        BaggageRegistration expected = expectedDetails.get(0);
        assertThat(baggageRegistration). isEqualTo(expected);
    }
}
```

Create a new service instance. ❶

Process the registration message. ❷

Check the outcomes. ❸

Here you create a new baggage registration service ❶ and use it to transform the message into a `BaggageRegistration` object ❷. Then you check that the generated registration object has the same values that you provided in the `Then` statement ❸.